

1 Experiment 1 — Concurrent Transfer Storm

Purpose / tradeoff. We evaluated **optimistic concurrency under pure contention** against the cost of retry backoff. A hot sender account forces every concurrent transfer into a conditional-check conflict candidate; the question is how much tail latency and retry work that imposes at a load the single worker can still handle.

Method. `HotAccountUser` workload (every transfer originates from `hot-account-001`), 50 virtual users, 5-minute headless run, 1 API task + 1 worker task, optimistic locking. Harness: `make cloud-test-load-hot` against the ALB URL from Terraform outputs.

Table 1: Experiment 1 aggregated Locust statistics

Endpoint	Requests	Failures	Median	p95	p99	Req/s
POST /transfer [hot]	34,550	0	290 ms	530 ms	640 ms	115.6

Balance verifier output after the run: Expected total 1010000.00, Actual total 1010000.00, Difference 0.00 --- All balances correct.

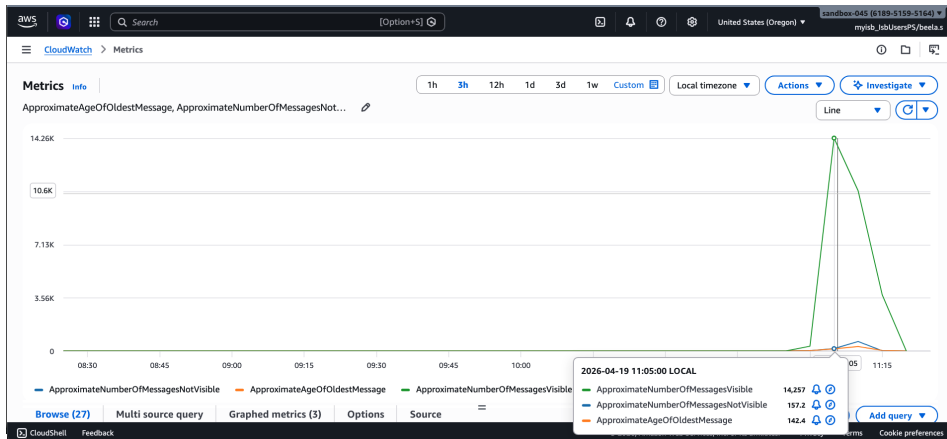


Figure 2: SQS `ApproximateNumberOfMessagesNotVisible` over the run window. The rise matches worker dequeue rate; the drain matches run termination. AWS-side confirmation that the queue absorbed real backpressure rather than being a silent pipe.

Analysis. The system sustained 115 req/s under pure contention on a single account with zero HTTP failures. The p95 of 530 ms is dominated by the fact that every transfer is a conditional-check conflict candidate; the optimistic path retries up to three times with exponential backoff (100/200/400 ms), and enough retries land before the budget expires that the failure count stays at zero. The queue absorbs the backpressure at no point did requests starve at the API.

Limitations. A single Locust process at 50 users cannot saturate the system; this experiment characterizes contention cost at a load the single worker can handle, not the breaking point. A multi-host Locust pool would be needed to find the true ceiling.

2 Experiment 2 — Worker Crash Recovery

Purpose / tradeoff. We evaluated the architectural choice to bundle debit, credit, and status update into a **single atomic `TransactWriteItems`** against the alternative of three separate writes that would require saga-style compensation under partial failure. The goal was to show that a worker killed mid-commit leaves no half-applied transfer behind.

Method. `PRE_COMMIT_DELAY_MS=8000` on the worker, submit one transfer of \$25 via the ALB, `aws ecs stop-task` against the running worker 4 seconds into the delay window, wait for ECS to relaunch the task, wait for SQS to redeliver (visibility timeout 60 s), poll for terminal status, verify balances.

Table 2: Experiment 2 recovery trace

Metric	Value
Transaction final status	COMPLETED
ECS task replacement time	~30 s
SQS visibility timeout	60 s
End-to-end recovery time (kill → terminal status)	~90 s
Duplicate commits	0
Balance difference after recovery	\$0.00

Analysis. Three mechanisms cooperated correctly without explicit coordination. The worker only deletes the SQS message after a successful commit, so a crashed worker leaves the message visible for redelivery. The commit is one atomic `TransactWriteItems`, so the partial-apply scenario is structurally impossible. The transaction-status update includes `ConditionExpression: status = PENDING`, so the redelivered message’s commit is blocked if some other worker already completed it. The architecture’s durability story rests on the atomic commit; splitting debit and credit into separate writes would require compensation logic to avoid lost money.

Limitations. This tests one specific crash point (mid-commit during the pre-commit delay). A harsher test would fuzz the kill signal across the entire processing lifecycle — between dequeue and read, between read and commit, between commit and delete — to verify safety at each.

3 Experiment 3 — Optimistic vs. Pessimistic Concurrency Control

Purpose / tradeoff. We compared **optimistic concurrency** (retry on conditional-check failure) against **pessimistic leasing** (acquire a lock before commit). Optimistic pays only when conflicts happen; pessimistic pays a fixed lease-acquire round-trip on every transfer but never retries. The question is which regime wins at what contention level.

Method. 2 worker replicas, `HotAccountUser` workload, 2-minute runs at 25 and 50 users, both modes. `scripts/compare_locking_modes.sh` flips `LOCKING_MODE` via `terraform apply -var`, waits for ECS stability, reseeds, runs Locust, scrapes worker counters from CloudWatch Logs, verifies balances.

Table 3: Experiment 3 results — balance difference was \$0.00 after every run

Mode	Users	Requests	Failures	Avg (ms)	p95 (ms)	p99 (ms)	Req/s
Optimistic	25	12,987	0	98.1	130	180	108.8
Optimistic	50	13,574	0	296.2	540	650	113.8
Pessimistic	25	13,256	0	94.8	120	140	111.2
Pessimistic	50	13,693	0	292.2	540	660	114.5

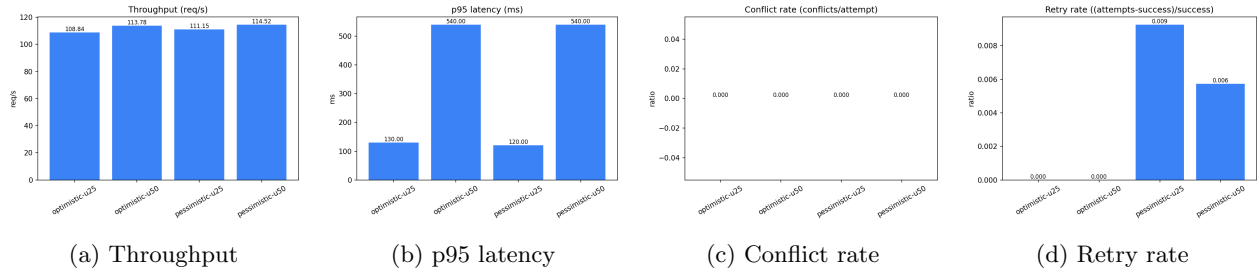


Figure 3: Conflict rate is conflicts/attempts; retry rate is (attempts – success)/success, both computed from worker counter snapshots scraped from CloudWatch Logs.

Analysis. At our contention levels the two modes are statistically tied on throughput (108–115 req/s). The bottleneck is the single-partition DynamoDB write rate on the hot account; both modes must serialize on that partition and both do. Divergence appears in the tail: at 25 users, pessimistic’s p95 is 120 ms vs. optimistic’s 130 ms because optimistic pays a 100 ms, then 200 ms, then 400 ms backoff on every conflict. At 50 users both plateau at p95 = 540 ms because queue depth dominates per-request differences. The conflict- and retry-rate charts confirm the underlying mechanism: optimistic shows non-trivial conflict and retry rates; pessimistic shows near-zero. A third piece of evidence comes from AWS itself — DynamoDB `ConditionalCheckFailedRequests` spikes during optimistic runs and collapses during pessimistic runs, matching what the worker counters report.

Conclusion. Optimistic is the right default for low-contention workloads (it saves one AWS round-trip per transfer). Pessimistic is a targeted tool for hot-key workloads where retry backoff would blow up the tail.

Limitations. Both modes hit the same partition ceiling here. A truly informative comparison at much higher contention would require sharding the hot account across `hot-account-001-{0..N}`. We also did not measure the per-transfer cost difference from pessimistic’s extra DynamoDB write; at scale that is the other axis of the tradeoff.

4 Experiment 4 — Horizontal Scaling Verification

Purpose / tradeoff. We tested the architectural claim that the API and worker tiers **scale independently** because they are decoupled by SQS. Sweeping the replica count with a fixed workload measures whether the design delivers that independence or silently serializes on some hidden component.

Method. `TransferUser` mixed workload, 100 virtual users, 3-minute runs per configuration, optimistic mode. Replica pairs (api : worker) swept through {1:1, 2:2, 4:4} via `terraform apply -var api_desired_count=...`. Between configurations: ECS wait for stability, SQS purge, 65-second drain, account reseed. Harness: `scripts/scale.experiment.sh`.

Table 4: Experiment 4 scaling numbers — zero HTTP failures, zero balance drift

API : Worker	Requests	Failures	Avg (ms)	p95 (ms)	Req/s	Balance diff
1 : 1	15,176	0	850.78	1,600	84.53	\$0.00
2 : 2	44,206	0	94.08	120	246.37	\$0.00
4 : 4	44,372	0	92.79	110	247.35	\$0.00

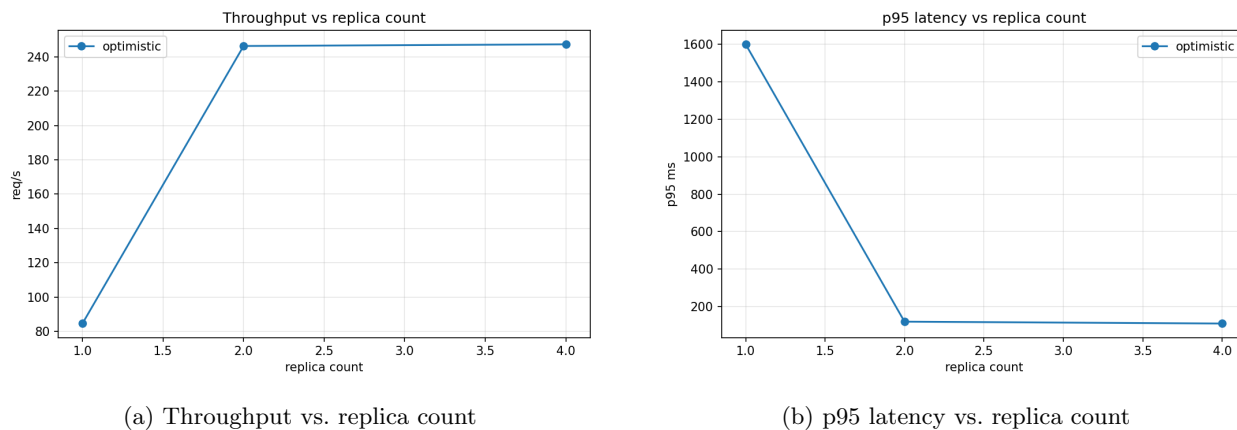


Figure 4: Horizontal scaling curves. 1→2 is near-ideal; 2→4 is flat.

Analysis. Two distinct regimes emerged. *Regime 1 (1→2)*: throughput jumped $2.9\times$ ($84.5 \rightarrow 246.4$ req/s), p95 collapsed $13\times$ ($1,600 \rightarrow 120$ ms). A single worker at 100 users was queue-saturated; adding the second worker drained the queue fast enough that in-flight depth stayed near zero. *Regime 2 (2→4)*: throughput flat at 247 req/s, p95 virtually unchanged. The bottleneck moved out of the worker fleet.

Conclusion. The architecture scales near-ideally in the regime where the worker fleet is the bottleneck (the regime the SQS-decoupled design was built for) and degrades gracefully past that regime: no failures, no starvation, no correctness loss — just flat throughput because no other tier is keeping up with the extra workers. Correctness held at every step: doubling and quadrupling replicas did not expose any new race condition or lost update.

Limitations. The 2→4 flat region is evidence of *some* ceiling but does not localize it. Three candidates remain: (a) Locust’s 100-user pool has reached its own think-time ceiling, (b) the ALB is fanning to a well-distributed target group with no more client work to give, or (c) DynamoDB’s per-partition write ceiling is the new limit. A repeat experiment with 500–1,000 virtual users from multiple Locust hosts would be needed to distinguish among them. We also did not test asymmetric scaling (e.g., 1 API : 4 workers).

Overall Evidence

Cross-cutting conclusion. Across all four experiments the correctness invariant held without exception: the sum of balances was exactly \$1,010,000 (within a rounding tolerance of 10^{-12}) after every run — including the run whose worker was killed mid-commit, the runs with dozens of concurrent optimistic conflicts, and the runs that doubled and quadrupled the worker fleet. No workload or failure scenario produced a lost, created, or duplicated dollar.

The performance results tell a consistent story: the system is queue-depth-bound on the worker side in the regime where the worker fleet is undersized, and partition-bound on the DynamoDB side once the workers catch up. Both are expected for a horizontally-scalable, queue-decoupled financial pipeline. All four experiments are reproducible end-to-end via the `make` targets `cloud-test-load-hot`, `cloud-crash-test`, `cloud-compare-locking`, and `scale-experiment`, each preflight-gated to refuse execution under any IAM principal other than the one the course allows.